

18-661 Introduction to Machine Learning

Linear Regression – II

Spring 2026

ECE – Carnegie Mellon University

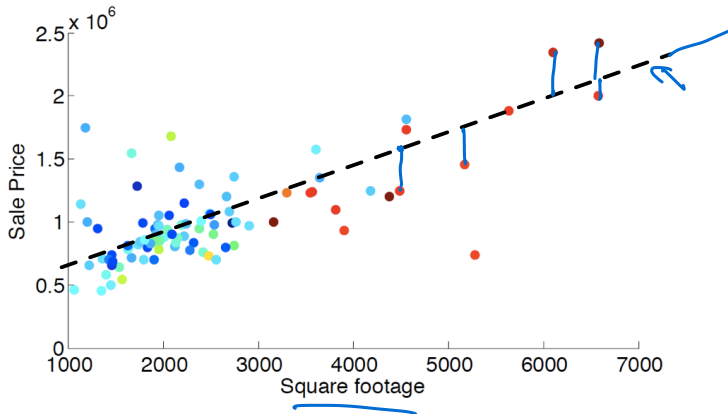
Announcements

- HW 1 is due on Friday, Jan 30th. Note that you can use up to 2 late days per homework, and up to 5 during the semester.
- First mini-exam is on Feb 9th, during the first half of the class. You are allowed to bring 1 one-sided handwritten US-letter-sized cheat sheet. No electronic devices are permitted. Calculators are allowed but will not be necessary.

1. Review of Linear Regression
2. Gradient Descent Methods
3. Feature Scaling
4. Ridge Regression
5. Non-linear Basis Functions

Review of Linear Regression

Example: Predicting House Prices



$$\text{Sale price} \approx \text{price_per_sqft} \times \text{square_footage} + \text{fixed_expense}$$

Minimize Squared Errors

Our model:

Sale_price =

price_per_sqft $\times$ square_footage + fixed_expense + {unexplainable_stuff}

Training data:

sqft	sale price	prediction	error	squared error
2000	810K	720K	90K	8100
2100	907K	800K	107K	107^2
1100	312K	350K	38K	38^2
5500	2,600K	2,600K	0	0
...	...			
Total				$8100 + 107^2 + 38^2 + 0 + \dots$

Aim:

Adjust price_per_sqft and fixed_expense such that the sum of the squared error is minimized — i.e., the unexplainable_stuff is minimized.

Linear Regression

Setup:

- **Input:** $\mathbf{x} \in \mathbb{R}^D$ (covariates, predictors, features, etc)
- **Output:** $y \in \mathbb{R}$ (responses, targets, outcomes, outputs, etc)
- **Model:** $f: \mathbf{x} \rightarrow y$, with $f(\mathbf{x}) = w_0 + \sum_{d=1}^D w_d x_d = w_0 + \mathbf{w}^\top \mathbf{x}$.
 - $\mathbf{w} = [w_1 \ w_2 \ \cdots \ w_D]^\top$: *weights, parameters, or parameter vector*
 - w_0 is called *bias*.
 - Sometimes, we also call $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \cdots \ w_D]^\top$ parameters.

Linear Regression

Setup:

- **Input:** $\mathbf{x} \in \mathbb{R}^D$ (covariates, predictors, features, etc)
- **Output:** $y \in \mathbb{R}$ (responses, targets, outcomes, outputs, etc)
- **Model:** $f : \mathbf{x} \rightarrow y$, with $f(\mathbf{x}) = w_0 + \sum_{d=1}^D w_d x_d = w_0 + \mathbf{w}^\top \mathbf{x}$.
 - $\mathbf{w} = [w_1 \ w_2 \ \cdots \ w_D]^\top$: *weights, parameters, or parameter vector*
 - w_0 is called *bias*.
 - Sometimes, we also call $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \cdots \ w_D]^\top$ parameters.
- **Training data:** $\mathcal{D} = \{(\underline{\mathbf{x}}_n, \underline{y}_n), n = 1, 2, \dots, N\}$

Linear Regression

Setup:

- **Input:** $\mathbf{x} \in \mathbb{R}^D$ (covariates, predictors, features, etc)
- **Output:** $y \in \mathbb{R}$ (responses, targets, outcomes, outputs, etc)
- **Model:** $f: \mathbf{x} \rightarrow y$, with $f(\mathbf{x}) = w_0 + \sum_{d=1}^D w_d x_d = w_0 + \mathbf{w}^\top \mathbf{x}$.
 - $\mathbf{w} = [w_1 \ w_2 \ \cdots \ w_D]^\top$: *weights, parameters, or parameter vector*
 - w_0 is called *bias*.
 - Sometimes, we also call $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \cdots \ w_D]^\top$ parameters.
- **Training data:** $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, 2, \dots, N\}$

Minimize the residual sum of squares:

$$\underline{RSS(\mathbf{w})} = \sum_{n=1}^N [y_n - f(\mathbf{x}_n)]^2 = \sum_{n=1}^N [y_n - (w_0 + \sum_{d=1}^D w_d x_{nd})]^2$$

over data samples

A Simple Case: $\mathbf{x}$ Is One-dimensional ($D=1$)

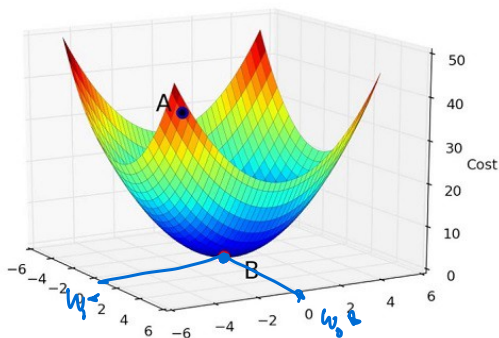
Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$

A Simple Case: x Is One-dimensional ($D=1$)

Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$



CONVEX function (has a unique global minimum w_0^*, w_1^*)

A Simple Case: x Is One-dimensional ($D=1$)

Residual sum of squares:

$$RSS(\mathbf{w}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$

Stationary points:

Take derivative with respect to parameters and set it to zero

$$\frac{\partial RSS(\mathbf{w})}{\partial w_0} = 0 \Rightarrow -2 \sum_n [y_n - (w_0 + w_1 x_n)] = 0,$$

$$\frac{\partial RSS(\mathbf{w})}{\partial w_1} = 0 \Rightarrow -2 \sum_n [y_n - (w_0 + w_1 x_n)] x_n = 0.$$

A Simple Case: $\mathbf{x}$ Is One-dimensional ($D=1$)

Residual sum of squares:

$$RSS(\mathbf{w}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$

Stationary points:

Take derivative with respect to parameters and set it to zero

$$\frac{\partial RSS(\mathbf{w})}{\partial w_0} = 0 \Rightarrow -2 \sum_n [y_n - (w_0 + w_1 x_n)] = 0,$$

$$\frac{\partial RSS(\mathbf{w})}{\partial w_1} = 0 \Rightarrow -2 \sum_n [y_n - (w_0 + w_1 x_n)] x_n = 0.$$

Solving the system we obtain the **least squares coefficient estimates**:

$$w_1 = \frac{\sum (x_n - \bar{x})(y_n - \bar{y})}{\sum (x_i - \bar{x})^2} \quad \text{and} \quad w_0 = \bar{y} - w_1 \bar{x}$$

where $\bar{x} = \frac{1}{N} \sum_n x_n$ and $\bar{y} = \frac{1}{N} \sum_n y_n$.

Least Mean Squares when $\mathbf{x}$ is D -dimensional

$RSS(\mathbf{w})$ in matrix form:

$$\underline{RSS(\tilde{\mathbf{w}})} = \sum_n [y_n - (\underline{w_0} + \sum_d w_d x_{nd})]^2 = \sum_n \underline{[y_n - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n]^2},$$

where we have redefined some variables (by augmenting)

$$\underline{\tilde{\mathbf{x}}} \leftarrow [1 \ x_1 \ x_2 \ \dots \ x_D]^\top, \quad \underline{\tilde{\mathbf{w}}} \leftarrow [w_0 \ w_1 \ w_2 \ \dots \ w_D]^\top$$

Least Mean Squares when $\mathbf{x}$ is D -dimensional

RSS($\mathbf{w}$) in matrix form:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - (w_0 + \sum_d w_d x_{nd})]^2 = \sum_n [y_n - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n]^2,$$

where we have redefined some variables (by augmenting)

$$\underline{\tilde{\mathbf{x}}} \leftarrow [1 \ x_1 \ x_2 \ \dots \ x_D]^\top, \quad \tilde{\mathbf{w}} \leftarrow [w_0 \ w_1 \ w_2 \ \dots \ w_D]^\top$$

Design matrix and target vector:

$$\underline{\tilde{\mathbf{X}}} = \begin{pmatrix} \tilde{\mathbf{x}}_1^\top \\ \tilde{\mathbf{x}}_2^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}, \quad \underline{\mathbf{y}} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Least Mean Squares when $\mathbf{x}$ is D -dimensional

RSS($\mathbf{w}$) in matrix form:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - (w_0 + \sum_d w_d x_{nd})]^2 = \sum_n [y_n - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n]^2,$$

where we have redefined some variables (by augmenting)

$$\tilde{\mathbf{x}} \leftarrow [1 \ x_1 \ x_2 \ \dots \ x_D]^\top, \quad \tilde{\mathbf{w}} \leftarrow [w_0 \ w_1 \ w_2 \ \dots \ w_D]^\top$$

Design matrix and target vector:

$$\tilde{\mathbf{X}} = \begin{pmatrix} \tilde{\mathbf{x}}_1^\top \\ \tilde{\mathbf{x}}_2^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Compact expression:

$$RSS(\tilde{\mathbf{w}}) = \|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Solution in Matrix Form

Compact expression

$$RSS(\tilde{\mathbf{w}}) = \|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Solution in Matrix Form

Compact expression

$$RSS(\tilde{\mathbf{w}}) = \|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \left\{ \underbrace{\tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}}} - 2 \underbrace{\left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top}_{\left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top} \tilde{\mathbf{w}} \right\} + \text{const}$$

Gradients of Linear and Quadratic Functions

- $\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A}\mathbf{x}$ (symmetric $\mathbf{A}$)

Solution in Matrix Form

Compact expression

$$RSS(\tilde{\mathbf{w}}) = ||\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}||_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Gradients of Linear and Quadratic Functions

- $\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A}\mathbf{x}$ (symmetric $\mathbf{A}$)

Normal equation

$$\nabla_{\tilde{\mathbf{w}}} RSS(\tilde{\mathbf{w}}) = \underline{2\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2\tilde{\mathbf{X}}^\top \mathbf{y}} = 0$$

Solution in Matrix Form

Compact expression

$$RSS(\tilde{\mathbf{w}}) = ||\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}||_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Gradients of Linear and Quadratic Functions

- $\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A} \mathbf{x}$ (symmetric $\mathbf{A}$)

Normal equation

$$\nabla_{\tilde{\mathbf{w}}} RSS(\tilde{\mathbf{w}}) = 2\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2\tilde{\mathbf{X}}^\top \mathbf{y} = 0$$

This leads to the **least-mean-squares** (LMS) solution

$$\tilde{\mathbf{w}}^{LMS} = \left(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$$

Why Minimize the RSS?

Probabilistic interpretation

- Noisy observation model for generating the dataset:

True value of y

Observed $\Rightarrow$ $Y = w_0 + w_1 X + \eta$ $\rightarrow$ noisy

where $\eta \sim N(0, \sigma^2)$ is a Gaussian random variable

Why Minimize the RSS?

Probabilistic interpretation

- **Noisy observation model** for generating the dataset:

$$Y = w_0 + w_1 X + \eta$$

where $\eta \sim N(0, \sigma^2)$ is a Gaussian random variable

- Conditional likelihood of one training sample:

$$\underline{p(y_n|x_n)} = N(w_0 + w_1 x_n, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{[y_n - (w_0 + w_1 x_n)]^2}{2\sigma^2}}$$

Probabilistic Interpretation (cont'd)

Log-likelihood of the training data $\mathcal{D}$ (assuming i.i.d):

$$\begin{aligned}\log P(\mathcal{D}) &= \log \prod_{n=1}^N p(y_n|x_n) = \sum_n \log p(y_n|x_n) \\ &= -\frac{1}{2} \left\{ \frac{1}{\sigma^2} \sum_n [y_n - (w_0 + w_1 x_n)]^2 + N \log \sigma^2 \right\} + \text{const}\end{aligned}$$

Probabilistic Interpretation (cont'd)

Log-likelihood of the training data $\mathcal{D}$ (assuming i.i.d):

$$\begin{aligned}\log P(\mathcal{D}) &= \log \prod_{n=1}^N p(y_n|x_n) = \sum_n \log p(y_n|x_n) \\ &= -\frac{1}{2} \left\{ \frac{1}{\sigma^2} \sum_n [y_n - (w_0 + w_1 x_n)]^2 + N \log \sigma^2 \right\} + \text{const}\end{aligned}$$

R_{SS} ↑

Estimating σ , w_0 and w_1 can be done in two steps

- Maximize over w_0 and w_1 :

$$\max \log P(\mathcal{D}) \Leftrightarrow \min \sum_n [y_n - (w_0 + w_1 x_n)]^2 \leftarrow \text{This is RSS}(\tilde{\mathbf{w}})!$$

- This gives a solid footing to our intuition: minimizing $\text{RSS}(\tilde{\mathbf{w}})$ is a sensible thing based on reasonable modeling assumptions.

Optimizing σ^2

$$\log P(\mathcal{D}) = -\frac{1}{2} \left\{ \frac{1}{\sigma^2} \sum_n [y_n - (w_0 + w_1 x_n)]^2 + N \log \sigma^2 \right\} + \text{const}$$

- Maximize over $s = \underline{\sigma^2}$:

$$\begin{aligned} \frac{\partial \log P(\mathcal{D})}{\partial s} &= -\frac{1}{2} \left\{ -\frac{1}{s^2} \sum_n [y_n - (w_0 + w_1 x_n)]^2 + N \frac{1}{s} \right\} = 0 \\ \rightarrow \underline{\sigma^{*2}} &= s^* = \frac{1}{N} \sum_n [y_n - (w_0 + w_1 x_n)]^2 \end{aligned}$$

- Estimating σ^* tells us how much noise there is in our predictions. For example, it allows us to place confidence intervals around our predictions.

5-min break for Gradescope quiz

Gradient Descent Methods

Review of Linear Regression

→ Gradient Descent Methods

Feature Scaling

Ridge Regression

Non-linear Basis Functions

Three Optimization Methods

Want to Minimize

$$\text{RSS}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} \right\} + \text{const}$$

-
- Least-Squares Solution; taking the derivative and setting it to zero
 - Batch Gradient Descent
 - Stochastic Gradient Descent

For simplicity of notation, we will replace the augmented parameter $\tilde{\mathbf{w}}$ with $\mathbf{w}$ and the augmented design matrix $\tilde{\mathbf{X}}$ with $\mathbf{X}$ from now on

Computational Complexity

Bottleneck of computing the solution?

$$\underline{w = \left(\underline{X^T X} \right)^{-1} X^T y}$$

How many operations do we need?

Computational Complexity

Bottleneck of computing the solution?

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

$$\mathbf{X} \in \mathbb{R}^{N \times (D+1)}$$

$$\mathbf{y} \in \mathbb{R}^{N \times 1}$$

How many operations do we need?

- $O(ND^2)$ for matrix multiplication $\mathbf{X}^\top \mathbf{X}$

Computational Complexity

Bottleneck of computing the solution?

$$\mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

How many operations do we need?

- $O(ND^2)$ for matrix multiplication $\mathbf{X}^\top \mathbf{X}$
- $O(D^3)$ (e.g., using Gauss-Jordan elimination) or $O(D^{2.373})$ (recent theoretical advances) for matrix inversion of $\mathbf{X}^\top \mathbf{X}$

Computational Complexity

Bottleneck of computing the solution?

$$\mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

How many operations do we need?

- $O(ND^2)$ for matrix multiplication $\mathbf{X}^\top \mathbf{X}$
- $O(D^3)$ (e.g., using Gauss-Jordan elimination) or $O(D^{2.373})$ (recent theoretical advances) for matrix inversion of $\mathbf{X}^\top \mathbf{X}$
- $O(ND)$ for matrix multiplication $\mathbf{X}^\top \mathbf{y}$

$$\begin{matrix} \mathbf{X}^\top & \mathbf{y} \\ 201 \times N & N \times 1 \end{matrix}$$

Computational Complexity

Bottleneck of computing the solution?

$$\mathbf{w} = \underbrace{(\mathbf{X}^\top \mathbf{X})^{-1}} \underbrace{\mathbf{X}^\top \mathbf{y}}$$

How many operations do we need?

- $O(ND^2)$ for matrix multiplication $\mathbf{X}^\top \mathbf{X}$
- $O(D^3)$ (e.g., using Gauss-Jordan elimination) or $O(D^{2.373})$ (recent theoretical advances) for matrix inversion of $\mathbf{X}^\top \mathbf{X}$
- $O(ND)$ for matrix multiplication $\mathbf{X}^\top \mathbf{y}$
- $O(D^2)$ for $(\mathbf{X}^\top \mathbf{X})^{-1}$ times $\mathbf{X}^\top \mathbf{y}$

Computational Complexity

Bottleneck of computing the solution?

$$\mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

How many operations do we need?

- $O(ND^2)$ for matrix multiplication $\mathbf{X}^\top \mathbf{X}$
- $O(D^3)$ (e.g., using Gauss-Jordan elimination) or $O(D^{2.373})$ (recent theoretical advances) for matrix inversion of $\mathbf{X}^\top \mathbf{X}$
- $O(ND)$ for matrix multiplication $\mathbf{X}^\top \mathbf{y}$
- $O(D^2)$ for $\left(\mathbf{X}^\top \mathbf{X} \right)^{-1}$ times $\mathbf{X}^\top \mathbf{y}$

Computational Complexity

Bottleneck of computing the solution?

$$\mathbf{w} = \underbrace{(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}}$$

How many operations do we need?

- $O(ND^2)$ for matrix multiplication $\mathbf{X}^\top \mathbf{X}$
- $O(D^3)$ (e.g., using Gauss-Jordan elimination) or $O(D^{2.373})$ (recent theoretical advances) for matrix inversion of $\mathbf{X}^\top \mathbf{X}$
- $O(ND)$ for matrix multiplication $\mathbf{X}^\top \mathbf{y}$
- $O(D^2)$ for $(\mathbf{X}^\top \mathbf{X})^{-1}$ times $\mathbf{X}^\top \mathbf{y}$

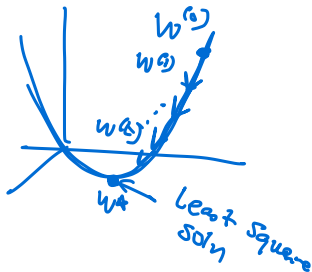
$O(ND^2)$ + $O(D^3)$ – Impractical for very large D or N

Alternative Method: Batch Gradient Descent

(Batch) Gradient Descent

- Initialize $\mathbf{w}$ to $\mathbf{w}^{(0)}$ (e.g., randomly);
set $t = 0$; choose $\eta > 0$

η
step size or learning rate



Alternative Method: Batch Gradient Descent

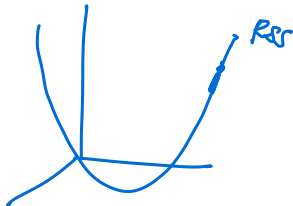
(Batch) Gradient Descent

- Initialize $\underline{\mathbf{w}}$ to $\underline{\mathbf{w}}^{(0)}$ (e.g., randomly);
set $t = 0$; choose $\eta > 0$
- Loop *until convergence*

Alternative Method: Batch Gradient Descent

(Batch) Gradient Descent

- Initialize $\mathbf{w}$ to $\mathbf{w}^{(0)}$ (e.g., randomly); set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
 1. Compute the gradient
$$\underline{\nabla \text{RSS}(\mathbf{w})} = \underline{\mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})}$$



Alternative Method: Batch Gradient Descent

(Batch) Gradient Descent

- Initialize $\mathbf{w}$ to $\mathbf{w}^{(0)}$ (e.g., randomly);
set $t = 0$; choose $\eta > 0$
- Loop *until convergence*

1. Compute the gradient

$$\nabla \text{RSS}(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$$

2. Update the parameters

$$\underline{\mathbf{w}^{(t+1)}} = \underline{\mathbf{w}^{(t)}} - \underline{\eta \nabla \text{RSS}(\mathbf{w})} \quad \leftarrow$$

Moving in the direction
of steepest descent

Alternative Method: Batch Gradient Descent

(Batch) Gradient Descent

- Initialize $\mathbf{w}$ to $\mathbf{w}^{(0)}$ (e.g., randomly);
set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
 1. Compute the gradient
$$\nabla \text{RSS}(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$$
 2. Update the parameters
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w})$$
 3. $t \leftarrow t + 1$

Alternative Method: Batch Gradient Descent

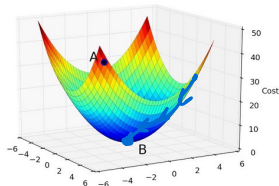
(Batch) Gradient Descent

- Initialize $\mathbf{w}$ to $\mathbf{w}^{(0)}$ (e.g., randomly);
set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
 1. Compute the gradient
$$\nabla \text{RSS}(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$$
 2. Update the parameters
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w})$$
 3. $t \leftarrow t + 1$

Alternative Method: Batch Gradient Descent

(Batch) Gradient Descent

- Initialize $\mathbf{w}$ to $\mathbf{w}^{(0)}$ (e.g., randomly); set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
 1. Compute the gradient
 $\nabla \text{RSS}(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$
 2. Update the parameters
 $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w})$
 3. $t \leftarrow t + 1$



What is the complexity of each iteration?

$O(ND)$ $\leftarrow$ Linear in N & D

Why Would This Work?

If gradient descent converges, it will converge to the same solution as using matrix inversion.

This is because $RSS(\mathbf{w})$ is a convex function in its parameters $\mathbf{w}$.

Why Would This Work?

If gradient descent converges, it will converge to the same solution as using matrix inversion.

This is because $RSS(\mathbf{w})$ is a convex function in its parameters $\mathbf{w}$.

Hessian of RSS

$$RSS(\mathbf{w}) = \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} + \text{const}$$

$$\Rightarrow \frac{\partial^2 RSS(\mathbf{w})}{\partial \mathbf{w} \mathbf{w}^\top} = \underline{2 \mathbf{X}^\top \mathbf{X}}$$

Why Would This Work?

If gradient descent converges, it will converge to the same solution as using matrix inversion.

This is because $RSS(\mathbf{w})$ is a convex function in its parameters $\mathbf{w}$.

Hessian of RSS

$$\begin{aligned} RSS(\mathbf{w}) &= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} + \text{const} \\ \Rightarrow \frac{\partial^2 RSS(\mathbf{w})}{\partial \mathbf{w} \mathbf{w}^\top} &= \underline{2\mathbf{X}^\top \mathbf{X}} \end{aligned}$$

$\mathbf{X}^\top \mathbf{X}$ is positive semidefinite, because for any $\mathbf{v}$

$$\underline{\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v}} = \underline{\|\mathbf{X}^\top \mathbf{v}\|_2^2} \geq 0$$

Three Optimization Methods

Want to Minimize

$$RSS(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} \right\} + \text{const}$$

- Least-Squares Solution; taking the derivative and setting it to zero
- • Batch Gradient Descent OCND)
- Stochastic Gradient Descent $\xi_j \quad D \sim 100$
 $N \sim 10^5$

Stochastic Gradient Descent (SGD)

Widrow-Hoff rule: update parameters using one example at a time

- Initialize $\mathbf{w}$ to some $\mathbf{w}^{(0)}$; set $t = 0$; choose $\eta > 0$
- Loop until convergence

1. Randomly choose a training sample $\mathbf{x}_t$
2. Compute its contribution to the gradient

Vector of size $(D+1) \times 1$

$$\rightarrow \underline{\mathbf{g}_t} = \underline{(\mathbf{x}_t^\top \mathbf{w}^{(t)} - y_t) \mathbf{x}_t}$$

Choose just 1 out of the N training examples

3. Update the parameters
 $\mathbf{w}^{(t+1)}$ = $\mathbf{w}^{(t)}$ - $\eta \mathbf{g}_t$
4. $t \leftarrow t + 1$

Stochastic Gradient Descent (SGD)

Widrow-Hoff rule: update parameters using one example at a time

- Initialize $\mathbf{w}$ to some $\mathbf{w}^{(0)}$; set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
 1. Randomly choose a training sample $\mathbf{x}_t$
 2. Compute its contribution to the gradient

$$\mathbf{g}_t = (\mathbf{x}_t^\top \mathbf{w}^{(t)} - y_t) \mathbf{x}_t$$

3. Update the parameters
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \mathbf{g}_t$$
4. $t \leftarrow t + 1$

How does the complexity per iteration compare with gradient descent?

Stochastic Gradient Descent (SGD)

Widrow-Hoff rule: update parameters using one example at a time

- Initialize $\mathbf{w}$ to some $\mathbf{w}^{(0)}$; set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
 1. Randomly choose a training sample $\mathbf{x}_t$
 2. Compute its contribution to the gradient

$$\mathbf{g}_t = (\mathbf{x}_t^\top \mathbf{w}^{(t)} - y_t) \mathbf{x}_t$$

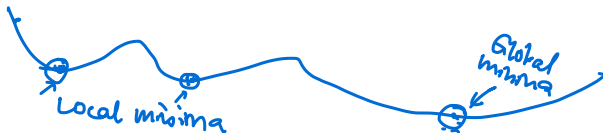
3. Update the parameters
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \mathbf{g}_t$$
4. $t \leftarrow t + 1$

How does the complexity per iteration compare with gradient descent?

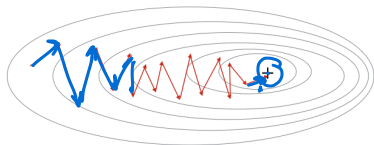
- $O(ND)$ for gradient descent versus $O(D)$ for SGD

SGD versus Batch GD

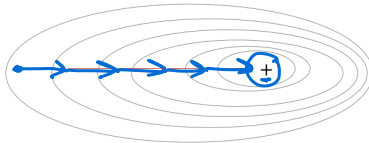
Non convex
obj



Stochastic Gradient Descent



Gradient Descent

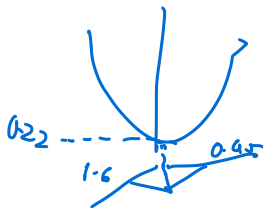


- SGD reduces per-iteration complexity from $O(ND)$ to $O(D)$
- But it is noisier and can take longer to converge

Example: Least Squares Solution

$N=4$ horses

<u>sqft (1000's)</u>	<u>sale price (100k)</u>
1	2
2	3.5
1.5	3
2.5	<u>4.5</u>



The w_0 and w_1 that minimize this are given by:

$$\underline{\mathbf{w}^{LMS}} = (\underline{\mathbf{X}^\top \mathbf{X}})^{-1} \underline{\mathbf{X}^\top \mathbf{y}}$$

$$\begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \left(\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 1.5 & 2.5 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 1.5 \\ 1 & 2.5 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 1.5 & 2.5 \end{bmatrix} \begin{bmatrix} 2 \\ 3.5 \\ 3 \\ 4.5 \end{bmatrix}$$

$$\begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} 0.45 \\ 1.6 \end{bmatrix}$$

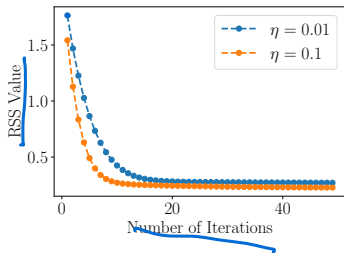
Minimum RSS is $\underline{RSS^*} = \|\mathbf{X}\mathbf{w}^{LMS} - \mathbf{y}\|_2^2 = \underline{0.2236}$

Example: Batch Gradient Descent

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w}) = \mathbf{w}^{(t)} - \eta \mathbf{X}^\top (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$$

Larger η gives faster convergence



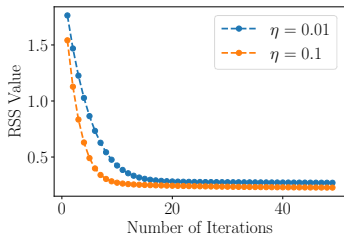
Example: Batch Gradient Descent

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

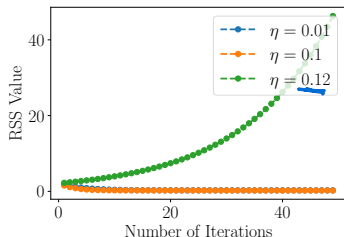
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w}) = \mathbf{w}^{(t)} - \eta \mathbf{X}^\top (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$$

True or the full gradient

Larger η gives faster convergence



But too large η makes GD unstable



Example: Stochastic Gradient Descent

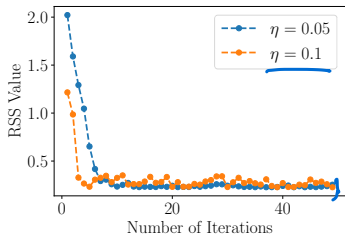
$N=4$

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

$$\underline{\mathbf{w}^{(t+1)}} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w}) = \mathbf{w}^{(t)} - \eta \left(\mathbf{x}_t^\top \mathbf{w}^{(t)} - y \right) \mathbf{x}_t$$

Noisy approx
of the true
gradient

Larger η gives faster convergence

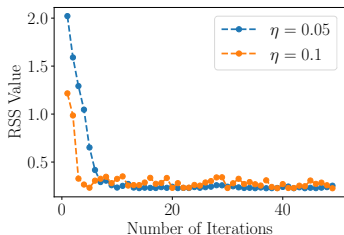


Example: Stochastic Gradient Descent

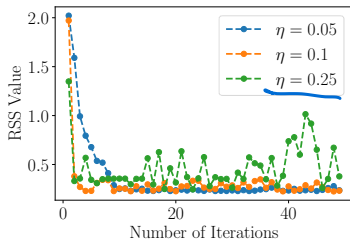
sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w}) = \mathbf{w}^{(t)} - \eta (\mathbf{x}_t^\top \mathbf{w}^{(t)} - \mathbf{y}) \mathbf{x}_t$$

Larger η gives faster convergence

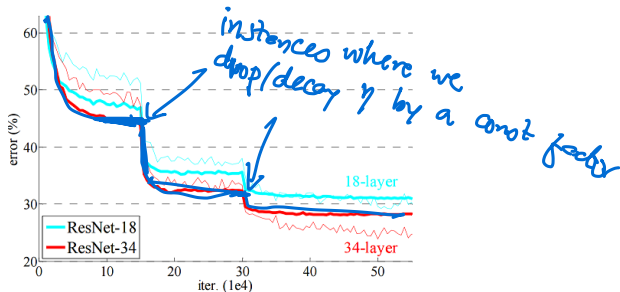


But too large η makes SGD unstable



How to Choose Learning Rate η in practice?

- Try 0.0001, 0.001, 0.01, 0.1 etc. on a validation dataset (more on this later) and choose the one that gives fastest, stable convergence
- Reduce η by a constant factor (eg. 10) when learning saturates so that we can reach closer to the true minimum.
- More advanced learning rate schedules such as AdaGrad, Adam, AdaDelta are used in practice.



Gradient Descent Methods in Machine Learning

- Batch gradient descent computes the exact gradient.
- Stochastic gradient descent approximates the gradient with a single data point; its expectation equals the true gradient.

Gradient Descent Methods in Machine Learning

Batch size
is N

- Batch gradient descent computes the exact gradient.

Batch
size = 1

- Stochastic gradient descent approximates the gradient with a single data point; its expectation equals the true gradient.

- Mini-batch variant: set the batch size to trade-off between accuracy of estimating gradient and computational cost

Batch size

$$1 \leq \underline{b} \leq N$$

Gradient Descent Methods in Machine Learning

- Batch gradient descent computes the exact gradient.
- Stochastic gradient descent approximates the gradient with a single data point; its expectation equals the true gradient.
- **Mini-batch** variant: set the batch size to trade-off between accuracy of estimating gradient and computational cost
- Similar ideas extend to other ML optimization problems.

Feature Scaling

Review of Linear Regression

Gradient Descent Methods



Feature Scaling

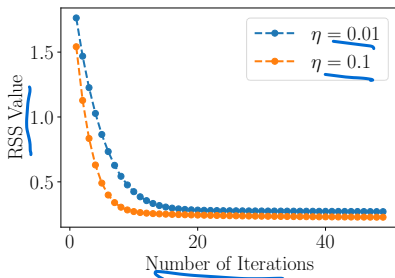
Ridge Regression

Non-linear Basis Functions

Batch Gradient Descent: Scaled Features

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \text{RSS}(\mathbf{w}) = \mathbf{w}^{(t)} - \eta \mathbf{X}^\top (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$$



Batch Gradient Descent: Without Feature Scaling

sqft	sale price
<u>1000</u>	<u>200,000</u>
<u>2000</u>	<u>350,000</u>
<u>1500</u>	<u>300,000</u>
<u>2500</u>	<u>450,000</u>

- Least-squares solution is $(w_0^*, w_1^*) = (45000, 160)$

Batch Gradient Descent: Without Feature Scaling

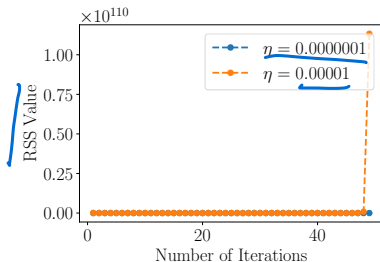
sqft	sale price
1000	200,000
2000	350,000
1500	300,000
2500	450,000

- Least-squares solution is $(w_0^*, w_1^*) = (45000, 160)$
- $\nabla \text{RSS}(\mathbf{w}^{(t)}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$ becomes HUGE, causing instability

Batch Gradient Descent: Without Feature Scaling

sqft	sale price
1000	200,000
2000	350,000
1500	300,000
2500	450,000

- Least-squares solution is $(w_0^*, w_1^*) = (45000, 160)$
- $\nabla \text{RSS}(\mathbf{w}^{(t)}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$ becomes HUGE, causing instability
- We need a tiny η to compensate, but this can cause numerical issues

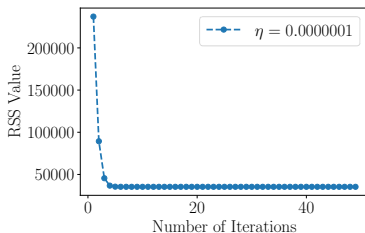


Batch Gradient Descent: Without Feature Scaling

² sqft	sale price
1000	200,000
2000	350,000
1500	300,000
2500	450,000

Min max scaled feature
 $\frac{1000 - 1000}{1500}$
 $\frac{2000 - 1000}{1500}$
 $\vdots$

- Least-squares solution is $(w_0^*, w_1^*) = (45000, 160)$
- $\nabla \text{RSS}(\mathbf{w})$ becomes HUGE, causing instability
- We need a tiny η to compensate, but this leads to slow convergence



How to Scale Features?

- Min-max normalization

$$x'_d = \frac{x_d - \min_n(x_d)}{\max_n x_d - \min_n x_d}$$

The min and max are taken over the possible values $x_d^{(1)}, \dots, x_d^{(N)}$ of x_d in the dataset. This will result in all scaled features $0 \leq x'_d \leq 1$

How to Scale Features?

- **Min-max normalization**

$$x'_d = \frac{x_d - \min_n(x_d)}{\max_n x_d - \min_n x_d}$$

The min and max are taken over the possible values $x_d^{(1)}, \dots, x_d^{(N)}$ of x_d in the dataset. This will result in all scaled features $0 \leq x_d \leq 1$

- **Mean normalization**

$$x'_d = \frac{x_d - \text{avg}(x_d)}{\max_n x_d - \min_n x_d}$$

This will result in all scaled features $-1 \leq x_d \leq 1$.

How to Scale Features?

- **Min-max normalization**

$$x'_d = \frac{x_d - \min_n(x_d)}{\max_n x_d - \min_n x_d}$$

The min and max are taken over the possible values $x_d^{(1)}, \dots, x_d^{(N)}$ of x_d in the dataset. This will result in all scaled features $0 \leq x'_d \leq 1$

- **Mean normalization**

$$x'_d = \frac{x_d - \text{avg}(x_d)}{\max_n x_d - \min_n x_d}$$

This will result in all scaled features $-1 \leq x'_d \leq 1$.

How to Scale Features?

- **Min-max normalization**

$$x'_d = \frac{x_d - \min_n(x_d)}{\max_n x_d - \min_n x_d}$$

The min and max are taken over the possible values $x_d^{(1)}, \dots, x_d^{(N)}$ of x_d in the dataset. This will result in all scaled features $0 \leq x_d \leq 1$

- **Mean normalization**

$$x'_d = \frac{x_d - \text{avg}(x_d)}{\max_n x_d - \min_n x_d}$$

This will result in all scaled features $-1 \leq x_d \leq 1$.

Labels $y^{(1)}, \dots, y^{(N)}$ should be similarly re-scaled

How to Scale Features?

- **Min-max normalization**

$$x'_d = \frac{x_d - \min_n(x_d)}{\max_n x_d - \min_n x_d}$$

The min and max are taken over the possible values $x_d^{(1)}, \dots, x_d^{(N)}$ of x_d in the dataset. This will result in all scaled features $0 \leq x_d \leq 1$

- **Mean normalization**

$$x'_d = \frac{x_d - \text{avg}(x_d)}{\max_n x_d - \min_n x_d}$$

This will result in all scaled features $-1 \leq x_d \leq 1$.

Labels $y^{(1)}, \dots, y^{(N)}$ should be similarly re-scaled

Several other methods: e.g., dividing by standard deviation (Z-score normalization)

good to manage outliers in the data

Ridge Regression

Review of Linear Regression

Gradient Descent Methods

Feature Scaling

Ridge Regression

Non-linear Basis Functions

What if $\mathbf{X}^\top \mathbf{X}$ Is Not Invertible?

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might this happen?

What if $X^T X$ Is Not Invertible?

$$\underline{X} \in \mathbb{R}^{N \times D+1}$$
$$X^T X \in \mathbb{R}^{(D+1) \times (D+1)}$$

$$\mathbf{w}^{LMS} = (\underline{X}^T \underline{X})^{-1} \underline{X}^T \mathbf{y}$$

Why might this happen?

- **Answer 1:** $N < \underline{D}$. Not enough data to estimate all parameters.
 $X^T X$ is not full-rank

What if $\mathbf{X}^\top \mathbf{X}$ Is Not Invertible?

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might this happen?

- **Answer 1:** $N < D$. Not enough data to estimate all parameters.
 $\mathbf{X}^\top \mathbf{X}$ is not full-rank
- **Answer 2:** Columns of $\mathbf{X}$ are not linearly independent, e.g., some features are linear functions of other features. In this case, solution is not unique. Examples:

What if $\mathbf{X}^\top \mathbf{X}$ Is Not Invertible?

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might this happen?

- **Answer 1:** $N < D$. Not enough data to estimate all parameters. $\mathbf{X}^\top \mathbf{X}$ is not full-rank
- **Answer 2:** Columns of $\mathbf{X}$ are not linearly independent, e.g., some features are linear functions of other features. In this case, solution is not unique. Examples:
 - A feature is a re-scaled version of another, for example, having two features correspond to length in meters and feet respectively

What if $\mathbf{X}^\top \mathbf{X}$ Is Not Invertible?

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might this happen?

- **Answer 1:** $N < D$. Not enough data to estimate all parameters.
 $\mathbf{X}^\top \mathbf{X}$ is not full-rank
- **Answer 2:** Columns of $\mathbf{X}$ are not linearly independent, e.g., some features are linear functions of other features. In this case, solution is not unique. Examples:
 - A feature is a re-scaled version of another, for example, having two features correspond to length in meters and feet respectively
 - Same feature is repeated twice (e.g., when there are many features)

What if $\mathbf{X}^\top \mathbf{X}$ Is Not Invertible?

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might this happen?

- **Answer 1:** $N < D$. Not enough data to estimate all parameters.
 $\mathbf{X}^\top \mathbf{X}$ is not full-rank
- **Answer 2:** Columns of $\mathbf{X}$ are not linearly independent, e.g., some features are linear functions of other features. In this case, solution is not unique. Examples:
 - A feature is a re-scaled version of another, for example, having two features correspond to length in meters and feet respectively
 - Same feature is repeated twice (e.g., when there are many features)
 - A feature has the same value for all data points
 - A feature is a linear combination of others, such as the sum of two features being equal to a third feature

Example: Matrix $X^T X$ Is Not Invertible

<u>sqft</u> (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
<u>2.5</u>	2	4.5

Design matrix and target vector:

$$\underline{\mathbf{X}} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 2 \\ 1 & 1.5 & 2 \\ 1 & 2.5 & 2 \end{bmatrix}, \quad \underline{\mathbf{w}} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}, \quad \underline{\mathbf{y}} = \begin{bmatrix} 2 \\ 3.5 \\ 3 \\ 4.5 \end{bmatrix}$$


rescaled versions
of each other

Example: Matrix $X^T X$ Is Not Invertible

sqft (1000's)	<u>bathrooms</u>	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

Design matrix and target vector:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 2 \\ 1 & 1.5 & 2 \\ 1 & 2.5 & 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 3.5 \\ 3 \\ 4.5 \end{bmatrix}$$

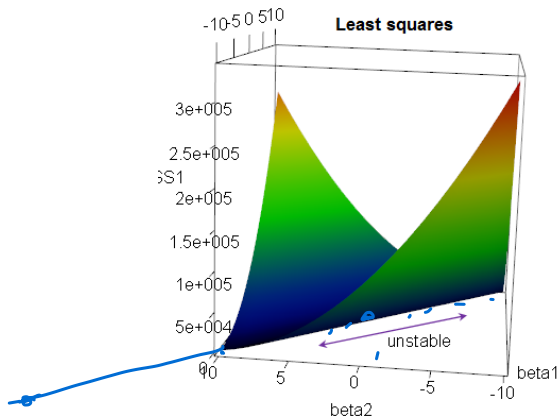
The 'bathrooms' feature is redundant, so we don't need w_2

$$\begin{aligned} y &= w_0 + w_1 x_1 + w_2 x_2 \\ &= w_0 + w_1 x_1 + w_2 \times 2 \quad \text{since } x_2 \text{ is always 2!} \\ y &= w_{0,\text{eff}} + w_1 x_1, \quad \text{where } w_{0,\text{eff}} = (w_0 + 2w_2) \end{aligned}$$

Eg $w_{0,\text{eff}} = 0.5$

What Does the RSS Look Like?

- When $\mathbf{X}^T \mathbf{X}$ is not invertible, the RSS objective function has a ridge, that is, the minimum is a line instead of a single point



$$\underline{w_0} + 2\underline{w_2} = w_{0,eff} = 0.45$$

In our example, this line is $w_{0,eff} = (w_0 + 2w_2)$

How Do You Fix This Issue?

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

- Manually remove redundant features

How Do You Fix This Issue?

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

- Manually remove redundant features
- But this can be tedious and non-trivial, especially when a feature is a linear combination of several other features

How Do You Fix This Issue?

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

- Manually remove redundant features
- But this can be tedious and non-trivial, especially when a feature is a linear combination of several other features

How Do You Fix This Issue?

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

- Manually remove redundant features
- But this can be tedious and non-trivial, especially when a feature is a linear combination of several other features

Need a general way that doesn't require manual feature engineering

SOLUTION: Ridge Regression

Ridge Regression

Intuition: what does a non-invertible $\mathbf{X}^\top \mathbf{X}$ mean?

Ridge Regression

Intuition: what does a non-invertible $\mathbf{X}^\top \mathbf{X}$ mean?

Consider the EVD (why does this exist?) of this matrix:

Ridge Regression

Intuition: what does a non-invertible $\mathbf{X}^\top \mathbf{X}$ mean?

Consider the EVD (why does this exist?) of this matrix:

$$\mathbf{X}^\top \mathbf{X} = \mathbf{V} \begin{bmatrix} \lambda_1 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ 0 & \cdots & \cdots & \cdots & 0 \\ 0 & \cdots & \cdots & \lambda_r & 0 \\ 0 & \cdots & \cdots & 0 & 0 \end{bmatrix} \mathbf{V}^\top$$

where $\lambda_1 \geq \lambda_2 \geq \cdots \lambda_r > 0$ and $r < D$. We will have a divide by zero issue when computing $(\mathbf{X}^\top \mathbf{X})^{-1} \dots$

Ridge Regression

Intuition: what does a non-invertible $\mathbf{X}^\top \mathbf{X}$ mean?

Consider the EVD (why does this exist?) of this matrix:

$$\mathbf{X}^\top \mathbf{X} = \mathbf{V} \begin{bmatrix} \lambda_1 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ 0 & \cdots & \cdots & \cdots & 0 \\ 0 & \cdots & \cdots & \lambda_r & 0 \\ 0 & \cdots & \cdots & 0 & 0 \end{bmatrix} \mathbf{V}^\top$$

where $\lambda_1 \geq \lambda_2 \geq \cdots \lambda_r > 0$ and $r < D$. We will have a divide by zero issue when computing $(\mathbf{X}^\top \mathbf{X})^{-1} \dots$

Fix the problem: ensure all singular values are non-zero:

$$\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} = \mathbf{V} \text{diag}(\lambda_1 + \lambda, \lambda_2 + \lambda, \cdots, \lambda) \mathbf{V}^\top$$

where $\lambda > 0$ and $\mathbf{I}$ is the identity matrix.

Regularized Least Squares (Ridge Regression)

Solution

$$\underline{w = \left(\underline{X^T X} + \lambda I \right)^{-1} X^T y}$$

If λ is very small we
don't change the soln too much

Regularized Least Squares (Ridge Regression)

Solution

$$\mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

This is equivalent to adding an extra term to $RSS(\mathbf{w})$

$$\overbrace{\frac{1}{2} \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \left(\mathbf{X}^\top \mathbf{y} \right)^\top \mathbf{w} + \text{const.} \right\}}^{RSS(\mathbf{w})} + \underbrace{\frac{1}{2} \lambda \|\mathbf{w}\|_2^2}_{\text{regularization}}$$
$$\frac{1}{2} \left\{ \mathbf{w}^\top \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} \right) \mathbf{w} - 2 \left(\mathbf{X}^\top \mathbf{y} \right)^\top \mathbf{w} + \text{const.} \right\}$$

Setting the
gradient to
0 will give us
this solution

Regularized Least Squares (Ridge Regression)

Solution

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

This is equivalent to adding an extra term to $RSS(\mathbf{w})$

Ridge Regression Obj =
$$\overbrace{\frac{1}{2} \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} + \text{const.} \right\}}^{RSS(\mathbf{w})} + \underbrace{\frac{1}{2} \lambda \|\mathbf{w}\|_2^2}_{\text{regularization}}$$

$$\nabla (\text{Ridge Regression Obj}) = \frac{1}{2} \left\{ \mathbf{w}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} + \text{const.} \right\} = 0$$

Benefits

- Numerically more stable, invertible matrix
- Force $\mathbf{w}$ to be small
- Prevent overfitting — more on this in the next lecture

Solve to get
$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Ridge Regression on Our Example

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

Ridge Regression on Our Example

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

The 'bathrooms' feature is redundant, so we don't need w_2

$$y = w_0 + w_1x_1 + w_2x_2$$

$$= w_0 + w_1x_1 + w_2 \times 2,$$

$$= w_{0,eff} + w_1x_1,$$

$$= \underline{0.45} + 1.6x_1$$

1

since x_2 is always 2!

where $w_{0,eff} = (w_0 + 2w_2)$

Should get this

Ridge Regression on Our Example

The 'bathrooms' feature is redundant, so we don't need w_2

$$\begin{aligned}y &= w_0 + w_1x_1 + w_2x_2 \\&= w_0 + w_1x_1 + w_2 \times 2, \quad \text{since } x_2 \text{ is always 2!} \\&= w_{0,eff} + w_1x_1, \quad \text{where } w_{0,eff} = \underline{(w_0 + 2w_2)} \\&= \underline{0.45} + 1.6x_1 \quad \text{Should get this}\end{aligned}$$

Ridge Regression on Our Example

The 'bathrooms' feature is redundant, so we don't need w_2

$$\begin{aligned}y &= w_0 + w_1 x_1 + w_2 x_2 \\&= w_0 + w_1 x_1 + w_2 \times 2, \quad \text{since } x_2 \text{ is always } 2! \\&= w_{0,eff} + w_1 x_1, \quad \text{where } w_{0,eff} = (w_0 + 2w_2) \\&= 0.45 + 1.6x_1 \quad \text{Should get this}\end{aligned}$$

Compute the solution for $\lambda = 0.5$

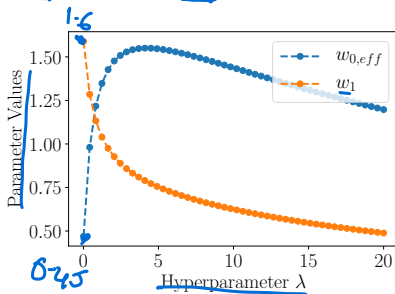
$$\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

$$\begin{bmatrix} \underline{w_0} \\ \underline{w_1} \\ \underline{w_2} \end{bmatrix} = \begin{bmatrix} \underline{0.208} \\ \underline{1.247} \\ \underline{0.4166} \end{bmatrix} \quad \text{recall} \quad \begin{bmatrix} w_{0,eff} \\ \underline{w_1} \end{bmatrix} = \begin{bmatrix} 0.45 \\ 1.6 \end{bmatrix} \quad \text{for LMS}$$

How Does λ Affect the Solution?

$$\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \underbrace{(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}}$$

Let us plot $w_{0,eff} = w_0 + 2w_2$ and w_1 for different $\lambda \in [0.01, 20]$

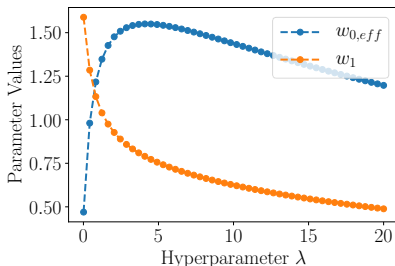


Setting small λ gives almost the least-squares solution, but it can cause numerical instability in the inversion

How to Choose λ ?

λ is referred to as a *hyperparameter*

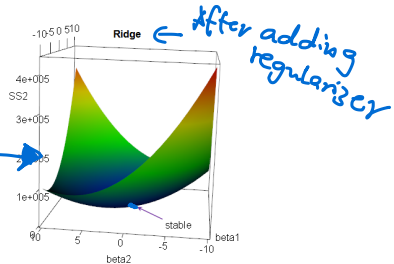
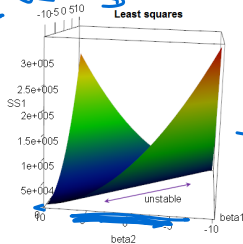
- Associated with the estimation method, not the dataset
- In contrast, $\mathbf{w}$ is the parameter vector
- Use validation set or cross-validation to find good choice of λ (more on this in the next lecture)



Why Is It Called Ridge Regression?

- When $\mathbf{X}^\top \mathbf{X}$ is not invertible, the RSS objective function has a **ridge**, that is, the minimum is a line instead of a single point
- Adding the regularizer term $\frac{1}{2} \lambda \|\mathbf{w}\|_2^2$ yields a unique minimum, thus avoiding instability in matrix inversion

Before adding
regularizer



Probabilistic Interpretation of Ridge Regression

Add a term to the objective function.

- Choose the parameters to not just minimize risk (i.e., minimize the RSS), but also avoid being too large.

$$\frac{1}{2} \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \left(\mathbf{X}^\top \mathbf{y} \right)^\top \mathbf{w} \right\} + \frac{1}{2} \lambda \|\mathbf{w}\|_2^2$$

Probabilistic Interpretation of Ridge Regression

Add a term to the objective function.

- Choose the parameters to not just minimize risk (i.e., minimize the RSS), but also avoid being too large.

$$\frac{1}{2} \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \left(\mathbf{X}^\top \mathbf{y} \right)^\top \mathbf{w} \right\} + \frac{1}{2} \lambda \|\mathbf{w}\|_2^2$$

Probabilistic Interpretation of Ridge Regression

Add a term to the objective function.

- Choose the parameters to not just minimize risk (i.e., minimize the RSS), but also avoid being too large.

$$\frac{1}{2} \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \left(\mathbf{X}^\top \mathbf{y} \right)^\top \mathbf{w} \right\} + \frac{1}{2} \lambda \|\mathbf{w}\|_2^2$$

Probabilistic interpretation: Place a prior on our weights

- Interpret $\mathbf{w}$ as a random variable
- Assume that each w_d is centered around zero
- Use observed data $\mathcal{D}$ to update our prior belief on $\mathbf{w}$

Probabilistic Interpretation of Ridge Regression

Add a term to the objective function.

- Choose the parameters to not just minimize risk (i.e., minimize the RSS), but also avoid being too large.

$$\frac{1}{2} \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \left(\mathbf{X}^\top \mathbf{y} \right)^\top \mathbf{w} \right\} + \frac{1}{2} \lambda \|\mathbf{w}\|_2^2$$

Probabilistic interpretation: Place a prior on our weights

- Interpret $\mathbf{w}$ as a random variable
- Assume that each w_d is centered around zero
- Use observed data $\mathcal{D}$ to update our prior belief on $\mathbf{w}$

Gaussian priors lead to ridge regression.

Review: Probabilistic Interpretation of Linear Regression

Linear Regression model: $Y = \mathbf{w}^\top \mathbf{X} + \eta$

$\eta \sim N(0, \sigma_0^2)$ is a Gaussian random variable and $Y \sim N(\mathbf{w}^\top \mathbf{X}, \sigma_0^2)$

Review: Probabilistic Interpretation of Linear Regression

Linear Regression model: $Y = \mathbf{w}^\top \mathbf{X} + \eta$
 $\eta \sim N(0, \sigma_0^2)$ is a Gaussian random variable and $Y \sim N(\mathbf{w}^\top \mathbf{X}, \sigma_0^2)$

Frequentist interpretation: We assume that $\mathbf{w}$ is fixed.

- The likelihood function maps parameters to probabilities

$$L : \mathbf{w}, \sigma_0^2 \mapsto p(\mathcal{D} | \mathbf{w}, \sigma_0^2) = p(\mathbf{y} | \mathbf{X}, \mathbf{w}, \sigma_0^2) = \prod_n p(y_n | \mathbf{x}_n, \mathbf{w}, \sigma_0^2)$$

- Maximizing the likelihood with respect to $\mathbf{w}$ minimizes the RSS and yields the LMS solution:

$$\mathbf{w}^{\text{LMS}} = \mathbf{w}^{\text{ML}} = \arg \max_{\mathbf{w}} L(\mathbf{w}, \sigma_0^2)$$

Ridge Regression model: $Y = \mathbf{w}^\top \mathbf{X} + \eta$

Probabilistic Interpretation of Ridge Regression

Ridge Regression model: $Y = \mathbf{w}^\top \mathbf{X} + \eta$

- $Y \sim N(\mathbf{w}^\top \mathbf{X}, \sigma_0^2)$ is a Gaussian random variable (as before)
- $w_d \sim N(0, \sigma^2)$ are i.i.d. Gaussian random variables (unlike before)
- Note that all w_d share the same variance σ^2

Probabilistic Interpretation of Ridge Regression

Ridge Regression model: $Y = \mathbf{w}^\top \mathbf{X} + \eta$

- $Y \sim N(\mathbf{w}^\top \mathbf{X}, \sigma_0^2)$ is a Gaussian random variable (as before)
- $w_d \sim N(0, \sigma^2)$ are i.i.d. Gaussian random variables (**unlike before**)
- Note that all w_d share the same variance σ^2
- To find $\mathbf{w}$ given data $\mathcal{D}$, compute the posterior distribution of $\mathbf{w}$:

$$\underline{p(\mathbf{w}|\mathcal{D})} = \frac{p(\mathcal{D}|\mathbf{w})\underline{p(\mathbf{w})}}{p(\mathcal{D})}$$

- Maximum a posterior (MAP) estimate:

$$\underline{\mathbf{w}^{\text{MAP}}} = \arg \max_{\mathbf{w}} p(\mathbf{w}|\mathcal{D}) = \arg \max_{\mathbf{w}} \underbrace{p(\mathcal{D}|\mathbf{w})}_{\text{Gaussian}} \underbrace{p(\mathbf{w})}_{\text{Gaussian}}$$

Estimating w

Let $\mathbf{x}_1, \dots, \mathbf{x}_N$ be i.i.d. with $y|\mathbf{w}, \mathbf{x} \sim N(\mathbf{w}^\top \mathbf{x}, \sigma_0^2)$; $w_d \sim N(0, \sigma^2)$.

Estimating w

Let $\mathbf{x}_1, \dots, \mathbf{x}_N$ be i.i.d. with $y|\mathbf{w}, \mathbf{x} \sim N(\mathbf{w}^\top \mathbf{x}, \sigma_0^2)$; $w_d \sim N(0, \sigma^2)$.

Joint likelihood of data and parameters (given σ_0, σ):

$$p(\underline{\mathcal{D}}, \underline{\mathbf{w}}) = \underbrace{p(\mathcal{D}|\mathbf{w})}_{\substack{n}} \underbrace{p(\mathbf{w})}_{\substack{d}} = \prod_n p(y_n|\mathbf{x}_n, \mathbf{w}) \prod_d p(w_d)$$

Estimating w

Let $\mathbf{x}_1, \dots, \mathbf{x}_N$ be i.i.d. with $y|\mathbf{w}, \mathbf{x} \sim N(\mathbf{w}^\top \mathbf{x}, \sigma_0^2)$; $w_d \sim N(0, \sigma^2)$.

Joint likelihood of data and parameters (given σ_0, σ):

$$p(\mathcal{D}, \mathbf{w}) = p(\mathcal{D}|\mathbf{w})p(\mathbf{w}) = \prod_n p(y_n|\mathbf{x}_n, \mathbf{w}) \prod_d p(w_d)$$

Plugging in the Gaussian PDF, we get:

$$\begin{aligned} \log p(\mathcal{D}, \mathbf{w}) &= \sum_n \log p(y_n|\mathbf{x}_n, \mathbf{w}) + \sum_d \log p(w_d) \\ &= - \underbrace{\frac{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}{2\sigma_0^2}}_{\text{RSS}} - \underbrace{\sum_d \frac{1}{2\sigma^2} w_d^2}_{\text{Regularizer}} + \text{const} \end{aligned}$$

Estimating $\mathbf{w}$

Let $\mathbf{x}_1, \dots, \mathbf{x}_N$ be i.i.d. with $y|\mathbf{w}, \mathbf{x} \sim N(\mathbf{w}^\top \mathbf{x}, \sigma_0^2)$; $w_d \sim N(0, \sigma^2)$.

Joint likelihood of data and parameters (given σ_0, σ):

$$p(\mathcal{D}, \mathbf{w}) = p(\mathcal{D}|\mathbf{w})p(\mathbf{w}) = \prod_n p(y_n|\mathbf{x}_n, \mathbf{w}) \prod_d p(w_d)$$

Plugging in the Gaussian PDF, we get:

$$\begin{aligned} \log p(\mathcal{D}, \mathbf{w}) &= \sum_n \log p(y_n|\mathbf{x}_n, \mathbf{w}) + \sum_d \log p(w_d) \\ &= -\underbrace{\frac{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}{2\sigma_0^2}}_{\text{RSS}} - \underbrace{\sum_d \frac{1}{2\sigma^2} w_d^2}_{\text{regularizer}} + \text{const} \end{aligned}$$

MAP estimate: $\mathbf{w}^{\text{MAP}} = \arg \max_{\mathbf{w}} \log p(\mathcal{D}, \mathbf{w})$

$$\mathbf{w}^{\text{MAP}} = \underset{\mathbf{w}}{\operatorname{argmin}} \left\{ \underbrace{\frac{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}{2\sigma_0^2}}_{\text{RSS}} + \underbrace{\frac{1}{2\sigma^2} \|\mathbf{w}\|_2^2}_{\text{regularizer}} \right\}$$

Maximum a Posteriori (MAP) Estimate

MAP Estimate:

$$\mathbf{w}^{\text{MAP}} = \operatorname{argmin}_{\mathbf{w}} \left\{ \frac{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}{2\sigma_0^2} + \frac{1}{2\sigma^2} \|\mathbf{w}\|_2^2 \right\}$$

After multiplying by $2\sigma_0^2$:

Maximum a Posteriori (MAP) Estimate

MAP Estimate:

$$\mathbf{w}^{\text{MAP}} = \operatorname{argmin}_{\mathbf{w}} \left\{ \frac{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}{2\sigma_0^2} + \frac{1}{2\sigma^2} \|\mathbf{w}\|_2^2 \right\}$$

After multiplying by $2\sigma_0^2$:

$$\mathbf{w}^{\text{MAP}} = \operatorname{argmin}_{\mathbf{w}} \left\{ \underbrace{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}_{\text{RSS}} + \underbrace{\frac{\sigma_0^2}{\sigma^2} \|\mathbf{w}\|_2^2}_{\text{regularizer}} \right\}$$

which is the same as our ridge regression formulation if we define

$\lambda = \sigma_0^2 / \sigma^2 > 0$.

Maximum a Posteriori (MAP) Estimate

MAP Estimate:

$$\mathbf{w}^{\text{MAP}} = \operatorname{argmin}_{\mathbf{w}} \left\{ \frac{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}{2\sigma_0^2} + \frac{1}{2\sigma^2} \|\mathbf{w}\|_2^2 \right\}$$

After multiplying by $2\sigma_0^2$:

$$\mathbf{w}^{\text{MAP}} = \operatorname{argmin}_{\mathbf{w}} \left\{ \underbrace{\sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2}_{\text{RSS}} + \frac{\sigma_0^2}{\sigma^2} \underbrace{\|\mathbf{w}\|_2^2}_{\text{regularizer}} \right\}$$

which is the same as our ridge regression formulation if we define $\lambda = \sigma_0^2/\sigma^2 > 0$. This extra term $\|\mathbf{w}\|_2^2$ is called **regularization/regularizer** and controls the magnitude of $\mathbf{w}$.

What Does the MAP Estimate Tell Us?

$$\mathcal{E}(\mathbf{w}) = \sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2 + \lambda \|\mathbf{w}\|_2^2$$

where $\lambda > 0$ is used to denote σ_0^2/σ^2 .

What Does the MAP Estimate Tell Us?

$$\mathcal{E}(\mathbf{w}) = \sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2 + \lambda \|\mathbf{w}\|_2^2$$

where $\lambda > 0$ is used to denote σ_0^2/σ^2

Intuitions

noise in data $\rightarrow$ Variance of prior

- If $\lambda \rightarrow +\infty$, then $\sigma_0^2 \gg \sigma^2$: the variance of noise is far greater than what our prior model can allow for $\mathbf{w}$. In this case, our prior model on $\mathbf{w}$ will force $\mathbf{w}$ to be close to zero. Numerically,

$$\underline{\mathbf{w}^{\text{MAP}}} \rightarrow \mathbf{0}$$

What Does the MAP Estimate Tell Us?

$$\mathcal{E}(\mathbf{w}) = \sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2 + \lambda \|\mathbf{w}\|_2^2$$

where $\lambda > 0$ is used to denote σ_0^2/σ^2 .

Intuitions

- If $\lambda \rightarrow +\infty$, then $\sigma_0^2 \gg \sigma^2$: the variance of noise is far greater than what our prior model can allow for $\mathbf{w}$. In this case, our prior model on $\mathbf{w}$ will force $\mathbf{w}$ to be close to zero. Numerically,

$$\mathbf{w}^{\text{MAP}} \rightarrow \mathbf{0}$$

- If $\lambda \rightarrow 0$, then we trust our data more. Numerically,

$$\underline{\mathbf{w}^{\text{MAP}}} \rightarrow \underline{\mathbf{w}^{\text{LMS}}} = \operatorname{argmin} \sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2$$

1. Review of Linear Regression
2. Gradient Descent Methods
3. Feature Scaling
4. Ridge Regression
5. Non-linear Basis Functions

→ Will cover in next lecture

Non-linear Basis Functions

Review of Linear Regression

Gradient Descent Methods

Feature Scaling

Ridge Regression

Non-linear Basis Functions

Should We Always Use a Linear Model?

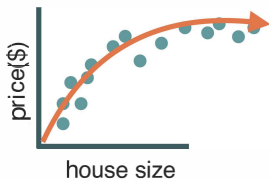


Figure 1: Sale price can saturate as square footage increases

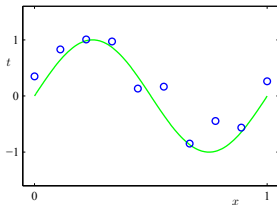


Figure 2: Temperature has cyclic variations over each year

We can use a nonlinear mapping to a new feature vector:

$$\phi(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^D \rightarrow \mathbf{z} \in \mathbb{R}^M$$

- M is dimensionality of new features $\mathbf{z}$ (or $\phi(\mathbf{x})$)
- M could be greater than, less than, or equal to D

General Nonlinear Basis Functions

We can use a nonlinear mapping to a new feature vector:

$$\phi(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^D \rightarrow \mathbf{z} \in \mathbb{R}^M$$

- M is dimensionality of new features $\mathbf{z}$ (or $\phi(\mathbf{x})$)
- M could be greater than, less than, or equal to D

We can apply existing learning methods on the transformed data:

- linear methods: prediction is based on $\mathbf{w}^\top \phi(\mathbf{x})$
- other methods: nearest neighbors, decision trees, etc

Residual sum of squares

$$\sum_n [\mathbf{w}^\top \phi(\mathbf{x}_n) - y_n]^2$$

where $\mathbf{w} \in \mathbb{R}^M$, the same dimensionality as the transformed features $\phi(\mathbf{x})$.

Residual sum of squares

$$\sum_n [\mathbf{w}^\top \phi(\mathbf{x}_n) - y_n]^2$$

where $\mathbf{w} \in \mathbb{R}^M$, the same dimensionality as the transformed features $\phi(\mathbf{x})$.

The LMS solution can be formulated with the new design matrix

$$\Phi = \begin{pmatrix} \phi(\mathbf{x}_1)^\top \\ \phi(\mathbf{x}_2)^\top \\ \vdots \\ \phi(\mathbf{x}_N)^\top \end{pmatrix} \in \mathbb{R}^{N \times M}, \quad \mathbf{w}^{\text{LMS}} = \left(\Phi^\top \Phi \right)^{-1} \Phi^\top \mathbf{y}$$

Example: Flexibility in Designing New Features!

x_1 , Area (1k sqft)	x_1^2 , Area ²	Price (100k)
1	1	2
2	4	3.5
1.5	2.25	3
2.5	6.25	4.5

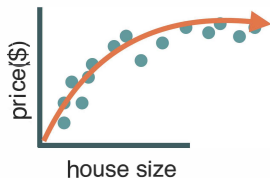


Figure 3: Add x_1^2 as a feature to allow us to fit quadratic, instead of linear functions of the house area x_1

Example: Flexibility in Designing New Features!

x_1 , front (100ft)	x_2 depth (100ft)	$10x_1x_2$, Lot (1k sqft)	Price (100k)
0.5	0.5	2.5	2
0.5	1	5	3.5
0.8	1.5	12	3
1.0	1.5	15	4.5

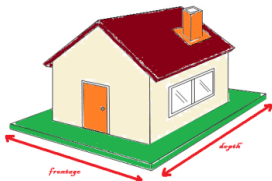


Figure 4: Instead of having frontage and depth as two separate features, it may be better to consider the lot-area, which is equal to $\text{frontage} \times \text{depth}$

Example with Regression

Polynomial basis functions

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{bmatrix} \Rightarrow f(x) = w_0 + \sum_{m=1}^M w_m x^m$$

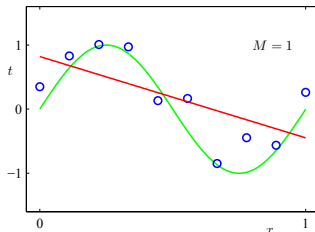
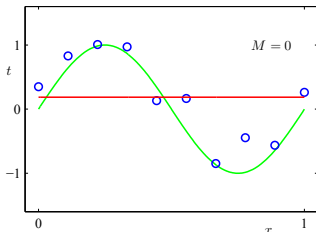
Example with Regression

Polynomial basis functions

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{bmatrix} \Rightarrow f(x) = w_0 + \sum_{m=1}^M w_m x^m$$

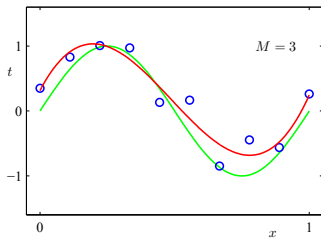
Fitting samples from a sine function:

underfitting since $f(x)$ is too simple



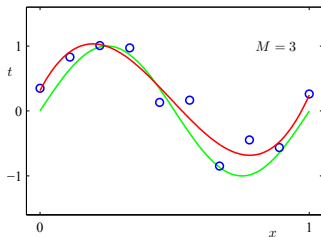
Adding Higher-order Terms

$M=3$

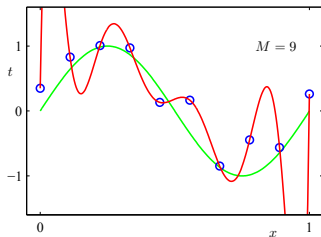


Adding Higher-order Terms

$M=3$



$M=9$: **overfitting**



More complex features lead to better results on the training data, but potentially worse results on new data, e.g., test data!

You Should Know

- Advantages and disadvantages of the least-mean-squares, batch gradient descent, and stochastic gradient descent solution methods
- Examples of feature scaling and why it can be important
- Formulation and solution of ridge regression
- Probabilistic interpretation of ridge regression
- How to use nonlinear basis functions in linear regression